Security Audit Report

Visha Oswal Community Platform — MVP Codebase

Generated: 16 March 2026

Executive Summary

A thorough manual review of the visha_oswal Next.js codebase was conducted, covering all API routes, authentication logic, database interactions, file upload handling, and client-side context. The application uses Next.js App Router, MongoDB/Mongoose, JWT authentication, AWS S3 for file uploads, and Google OAuth.

12 security issues were identified across 4 severity levels. The most critical findings require immediate attention before any public launch:

CRITICAL	HIGH	MEDIUM	LOW
4	3	3	2

The highest priority issues are: an unauthenticated admin endpoint (anyone can create events), XSS risk in the Google OAuth redirect flow, JWT tokens stored in localStorage (vulnerable to XSS theft), and zero input validation on experience/education data submission.

Scope & Methodology

Files reviewed: all API route handlers, authentication and DB utilities, S3 service, AppContext, and key models. The audit focused on:

- Authentication & authorization (JWT handling, route protection)
- Input validation and sanitization
- Data exposure and privacy (what is returned to unauthenticated callers)
- File upload security
- Common web vulnerabilities (XSS, injection, CSRF, IDOR)
- Security configuration (headers, rate limiting, error handling)

Findings

Critical Severity

1. XSS via token injection in HTML response CRITICAL

File: src/app/api/auth/google/route.ts

Description:

The Google OAuth redirect handler builds an HTML page with the JWT token embedded directly in a <script> tag using template literal interpolation: localStorage.setItem('token', '${token}'). If the JWT payload or any derived value somehow contained a single quote or script-closing tag, this would break out of the string. More critically, this pattern of server-rendering a JWT into raw HTML is fragile and sets a dangerous precedent.

Vulnerable pattern:

```
return new Response(`<script>localStorage.setItem('token', '${token}');</script>`)
```

Recommended Fix:

Use a more secure token delivery mechanism. Instead of injecting the token into HTML, redirect to a frontend callback URL with a short-lived code, or set an HttpOnly cookie. At minimum, JSON-encode and HTML-escape the token before embedding: JSON.stringify(token) and replace </script> with <\/script>.

---

2. Unauthenticated admin endpoint — anyone can create events CRITICAL

File: src/app/api/admin/events/route.ts

Description:

The admin event creation endpoint has no authentication or authorization check at all. The comment says 'Admin auth will be integrated later' — but this endpoint is live and publicly accessible right now. Any anonymous user on the internet can POST to /api/admin/events and create events in your database.

Vulnerable pattern:

```
// No verifyAuth call anywhere in this route handler
```

Recommended Fix:

Add verifyAuth() immediately, plus an admin role check. Create an isAdmin field on your User model and verify it: const user = verifyAuth(req); const dbUser = await User.findById(user.id); if (!dbUser?.isAdmin) return 401. Do not deploy admin endpoints without auth, even in MVP.

---

3. JWT stored in localStorage — vulnerable to XSS theft CRITICAL

File: src/context/AppContext.tsx + google/route.ts

Description:

The JWT token is stored in localStorage and read from localStorage on every request. localStorage is accessible to any JavaScript running on the page, meaning a single XSS vulnerability anywhere in your app (a third-party script, a user-injected comment, etc.) can silently steal all user tokens. This is especially risky for a community platform where users can post content.

Vulnerable pattern:

```
const token = localStorage.getItem('token'); // Accessible to any JS on the page
```

Recommended Fix:

Store the JWT in an HttpOnly, Secure, SameSite=Strict cookie instead. HttpOnly cookies cannot be read by JavaScript at all — they are automatically attached to requests by the browser. Update your login endpoints to Set-Cookie the token, and remove all localStorage.getItem/setItem('token') calls from the frontend.

---

4. No input validation or sanitization on user-supplied content CRITICAL

File: src/app/api/experience/route.ts, education/route.ts, member-profile/route.ts

Description:

The experience and education endpoints accept the entire request body (newExp, newEdu) and push it directly into the database with user.experience.unshift(newExp) — with zero validation. A malicious user can send arbitrary fields, inject unexpected data shapes, or attempt prototype pollution. The member-profile PUT similarly spreads personal data directly into $set with no field-level sanitization.

Vulnerable pattern:

```
user.experience.unshift(newExp); // newExp is raw req.json() with no validation
```

Recommended Fix:

Always whitelist the exact fields you accept. Never spread or unshift a raw request body into a database document. Define a strict schema: const newExp = { title: body.title, company: body.company, ... } and validate each field (required, type, length limits) before saving. Use a validation library like zod or joi.

High Severity

5. No rate limiting on auth endpoints — brute force attack possible HIGH

File: src/app/api/auth/login/route.ts, register/route.ts

Description:

The login and registration endpoints have no rate limiting. An attacker can send thousands of login attempts per second to brute-force passwords, enumerate registered email addresses via different error responses, or flood registrations to spam your database. This is especially critical since this is a community platform with real user accounts.

Vulnerable pattern:

```
// No rate limiting — this endpoint can be called unlimited times
```

Recommended Fix:

Add rate limiting using the upstash/ratelimit library or a Next.js middleware approach. Limit login attempts to ~5 per IP per minute. Also consider adding account lockout after N failed attempts, and ensure both valid and invalid email addresses return the same response time (timing-safe comparison) to prevent user enumeration.

---

6. Unvalidated file type in upload URL generation HIGH

File: src/app/api/upload-url/route.ts

Description:

The upload URL endpoint accepts any fileType from the query string and passes it directly to S3 as the ContentType, and uses it to determine the file extension. There is no validation that the fileType is actually an image. An attacker could request a presigned URL for application/javascript or text/html and upload malicious files to your S3 bucket.

Vulnerable pattern:

```
const extension = fileType.split('/')[1] || 'jpg'; // fileType comes directly from
query param
```

Recommended Fix:

Whitelist allowed MIME types explicitly: const ALLOWED_TYPES = ['image/jpeg','image/png','image/webp','image/gif']; if (!ALLOWED_TYPES.includes(fileType)) return 400. Also validate the uploadType parameter against an allowlist ['post','banner','profile'] rather than using it directly.

7. Search endpoint unauthenticated — exposes all member personal data HIGH

File: src/app/api/users/search/route.ts

Description:

The search endpoint has no authentication check. Any anonymous visitor can search your entire member database and retrieve names, usernames, profile pictures, city of origin, current city, and full personal profile objects. For a community platform with private member data, this is a serious privacy and security issue.

Vulnerable pattern:

```
// GET /api/users/search — no verifyAuth() call
```

Recommended Fix:

Add verifyAuth() to the search endpoint. Only authenticated, verified community members should be able to search for other members. Also review what the $project stage returns — the personal field includes a lot of sensitive info; consider projecting only the minimal fields needed for search result cards.

Medium Severity

8. JWT token expiry too short with no refresh mechanism MEDIUM

File: src/app/api/auth/login/route.ts, google/route.ts

Description:

JWTs are signed with a 5-hour expiry and there is no refresh token mechanism. When the token expires, users are silently logged out with no graceful recovery. This leads to poor UX and often pushes developers to increase token TTL to days or weeks — which increases the blast radius if a token is stolen.

Vulnerable pattern:

```
jwt.sign(payload, secret, { expiresIn: '5h' }) // No refresh mechanism
```

Recommended Fix:

Implement a refresh token pattern: issue a short-lived access token (15-30 min) and a long-lived refresh token (7-30 days) stored in an HttpOnly cookie. Add a /api/auth/refresh endpoint that validates the refresh token and issues a new access token. This balances security and UX.

---

9. Sensitive user data exposed in public profile endpoint MEDIUM

File: src/app/api/users/profile/[username]/route.ts

Description:

The public profile endpoint is unauthenticated (no verifyAuth) yet returns extended profile data including family relationships, city of origin, marital status, and gender. While contact number and DOB are excluded, a public endpoint returning family tree data for any username is a significant privacy risk for a community platform.

Vulnerable pattern:

```
// No verifyAuth() — family tree data returned to anyone
```

Recommended Fix:

Either require authentication to view profiles, or implement a tiered privacy system where members control what is public vs members-only. At minimum, add a requireAuth guard and return a slimmer public profile for unauthenticated requests. Consider adding a privacySettings field to MemberProfile.

10. Error messages reveal internal implementation details MEDIUM

File: Multiple API routes

Description:

Several error handlers return error.message directly in API responses (e.g., error: error?.message in notifications/route.ts). Internal error messages can leak database schema details, file paths, library versions, or stack traces to attackers. The console.error(err) calls in login/route.ts also log full error objects which may include sensitive data.

Vulnerable pattern:

```
return NextResponse.json({ error: error?.message }, { status: 500 }) // Leaks internals
```

Recommended Fix:

Always return generic error messages to clients: { error: 'Internal server error' }. Log the full error server-side only. Create an error handling utility that maps known error types to safe user-facing messages. Never expose error.message, error.stack, or database error codes directly in API responses.

Low Severity

11. Missing HTTP security headers LOW

File: next.config.mjs

Description:

The Next.js config has no security headers configured. Missing headers include: Content-Security-Policy (prevents XSS), X-Frame-Options (prevents clickjacking), X-Content-Type-Options (prevents MIME sniffing), Referrer-Policy, and Permissions-Policy. These are easy wins that significantly improve the security posture.

Vulnerable pattern:

```
// next.config.mjs has no headers() configuration
```

Recommended Fix:

Add a headers() function to next.config.mjs with recommended security headers. Next.js has documentation on this. At minimum add: X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin, and a Content-Security-Policy appropriate for your app.

12. No pagination limit enforcement on posts feed LOW

File: src/app/api/posts/route.ts

Description:

The posts GET endpoint fetches all posts with Post.find().sort().populate() with no limit or pagination. As the platform grows, this query will fetch the entire posts collection on every feed load, leading to memory exhaustion, slow responses, and potential denial-of-service. The search endpoint does paginate, so the pattern is inconsistently applied.

Vulnerable pattern:

```
const posts = await Post.find().sort({ createdAt: -1 }).populate(...) // No .limit()
```

Recommended Fix:

Add pagination to the posts feed: accept page and limit query params (with a max limit of 20-50), use .skip() and .limit() in the query, and return pagination metadata in the response. Also add an index on createdAt in your Post model for efficient sorting.

Priority Action Plan

Address issues in this order before launch:

Immediate (before any testing with real users)

- **Add verifyAuth() + admin role check to /api/admin/events**
- **Replace localStorage token storage with HttpOnly cookies**
- **Fix XSS risk in Google OAuth HTML redirect response**
- **Add input validation to experience, education, and member-profile endpoints**

Before launch

- Add rate limiting to /api/auth/login and /api/auth/register
- Whitelist MIME types in /api/upload-url
- Add verifyAuth() to /api/users/search

Post-launch improvements

- Implement refresh token mechanism
- Add profile privacy controls for public vs member-only data
- Standardize error responses to never expose internals
- Add Next.js security headers in next.config.mjs
- Add pagination + limit to posts feed query

Overall the codebase shows good structure and many routes are correctly protected with verifyAuth(). The ownership checks on post delete/edit (comparing post.user to authenticated user.id) are well implemented. The issues found are common MVP-stage oversights that are straightforward to fix.